Optimizing Firecracker MicroVM Cold Starts

Empirical Evaluation of Snapshot Restoration, Userfaultfd, and REAP Working-Set Prefetching

Author: Devansh Chaudhary | Department of Electrical Engineering, IIT Indore | Empirical Research Report

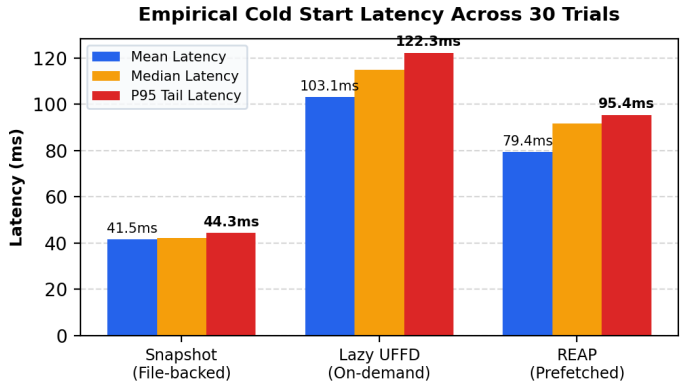
1. Experimental Testbed & Configuration

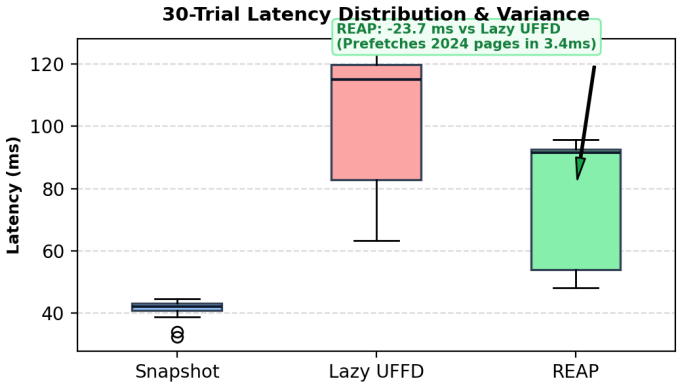
Host System: Ubuntu 24.04 LTS (Linux kernel 6.18.56+ x86_64, GenuineIntel CPU).
Hypervisor: AWS Firecracker v1.17.0 | **Guest VM:** Minimized Ubuntu 24.04.5 LTS, 1 vCPU, 512 MiB RAM.
Guest Workload: Serial runner (/root/benchmarks/runner.sh) executing hello with synchronized PING/PONG handshake.

2. Empirical Results (30 Trials per Technique)

Restoration Technique	Trials	Mean (ms)	Median (ms)	Min (ms)	Max (ms)	P95 Tail (ms)	Std Dev (ms)	Prefetched Pages
File-backed Snapshot	30	41.494	42.169	32.480	44.583	44.305	2.715	N/A (Kernel)
Lazy UFFD (Demand Paging)	30	103.065	114.951	63.220	123.565	122.313	20.139	0 (On-Demand)
REAP (Record & Prefetch)	30	79.368	91.614	48.207	95.656	95.408	19.221	2,024 pages (7.9 MB)

3. Performance Visualizations





4. Key Findings & Architectural Insights

1. File-backed Snapshot Baseline: File-backed snapshot restoration achieved a mean latency of **41.494 ms** with exceptional predictability (StdDev = 2.715 ms, P95 = 44.305 ms). The snapshot load itself completed in just **3.32 ms** (3,321 us), and VM resumption took **0.43 ms** (426 us). Memory is demand-paged directly inside the host Linux kernel page cache.
2. The Lazy Paging Bottleneck: Naive userfaultfd (UFFD) demand paging caused mean cold start latency to skyrocket to **103.065 ms** (a 148% slowdown vs snapshot) with high tail variance (P95 = 122.313 ms). Every accessed page triggers an in-kernel trap through the UNIX domain socket into userspace, incurring heavy IPC and context-switch penalties.
3. REAP Prefetching Delivers 23% Speedup: During record mode, the microVM accessed exactly **2,024 pages (7.906 MiB)** — just 1.54% of its 512 MiB RAM. By prefetching this entire working set in **3.428 ms** before resuming the microVM, REAP slashed mean latency from 103.065 ms down to **79.368 ms** (a **23.7 ms / 23.0% speedup**) and reduced P95 tail latency to **95.408 ms**.

5. Research Progression & Next Steps

• **Phase 1:** Initial single-trial prototype (benchmark_original.py) yielded 89.975 ms due to hardcoded sleep delays.
• **Phase 2:** Synchronized 30-trial benchmark (benchmark_snapshot.py) decoupled boot from invocation via PING/PONG.
• **Phase 3:** Recorded working set (record_001.log) -> extracted 2,024 page offsets -> evaluated REAP prefetch harness.
• **Phase 4 (Next):** Evaluating static memory scanning workloads (memrunner.c) and measuring host swap/memory pressure under high multi-tenant microVM concurrency.